P.Vishwa Nandan(192473015)

TASK 10 – Enforce Encapsulation through Packages and Access Modifiers

Marks: 3

This task is about **Encapsulation in Java**. Encapsulation means protecting the data of a class by keeping its fields private and providing controlled access through getter and setter methods.

The assignment requires:

1. Package structure.
2. Access modifiers.
3. Getters and setters with validation.
4. Wrapper classes and autoboxing.
5. Alternative approach using public fields and complexity analysis.

1. Package Structure

Packages are used to organize related Java classes.

For our Campus Smart-Parking System, we can create the following packages:

```
parking
  |
  |--- model
  |     |
  |     |--- Vehicle.java
  |     |--- ParkingSlot.java
  |
  |--- service
  |     |
  |     |--- ParkingService.java
  |
  |--- app
        |
        |--- Main.java
```

Explanation:

1. parking.model

Contains classes that represent parking entities, such as Vehicle and ParkingSlot.

2. parking.service

Contains classes that perform parking operations, such as allocating and releasing slots.

3. parking.app

Contains the Main class used to execute the parking application.

Packages improve code organization, reusability and maintainability.

2. Access Modifiers

Access modifiers control the visibility of classes, variables, constructors and methods.

Java provides four main access modifiers.

A. Private

- Accessible only within the same class.
- Used to protect important data.
- Example: private String regNo;

B. Public

- Accessible from anywhere.
- Used for methods and constructors that need to be accessed by other classes.
- Example: public void display();

C. Protected

- Accessible within the same package and by subclasses.
- Useful when child classes need access to parent-class members.

D. Default

- Used when no access modifier is specified.
- Accessible only within the same package.
- Useful for package-specific methods.

Access Modifier Selection

In our parking system:

- Fields are declared private to protect data.

- Constructors are public to allow object creation from other packages.
- Getters and setters are public to provide controlled access.
- Protected methods are used when subclasses need access.
- Default access is used for internal package-level operations.

3. Getters and Setters with Validation

Getters and setters are methods used to access and modify private variables.

A getter returns the value of a variable.

A setter updates the value of a variable after checking whether the given data is valid.

Example:

```
class Vehicle {

   private String regNo;
   private int parkingDuration;

   public String getRegNo() {
      return regNo;
   }

   public void setRegNo(String regNo) {

      if (regNo == null || regNo.isEmpty()) {
         System.out.println("Invalid vehicle number");
         return;
      }

      this.regNo = regNo;
   }

   public int getParkingDuration() {
```

```
        return parkingDuration;
    }

    public void setParkingDuration(int duration) {

        if (duration < 0) {
            System.out.println("Invalid parking duration");
            return;
        }

        parkingDuration = duration;
    }
}
```

Explanation:

- private prevents direct access to variables.
- getRegNo() returns the vehicle registration number.
- setRegNo() checks whether the registration number is valid.
- setParkingDuration() rejects negative parking duration.

Validation ensures that invalid data cannot easily enter the object.

4. Wrapper Classes and Autoboxing

Wrapper classes convert primitive data types into objects.

Examples:

- int → Integer
- double → Double
- char → Character
- boolean → Boolean

Autoboxing means automatically converting a primitive value into its corresponding wrapper object.

Example:

int duration = 120;

Integer parkingTime = duration;

Here, Java automatically converts the int value into an Integer object.

This is called autoboxing.

Unboxing means converting a wrapper object into a primitive value.

Example:

Integer parkingTime = 120;

int duration = parkingTime;

Here, Java automatically converts Integer into int.

Risk of Using == with Integer Objects

Consider:

Integer a = 1000;

Integer b = 1000;

System.out.println(a == b);

System.out.println(a.equals(b));

The == operator compares object references, whereas equals() compares their values.

Therefore, using equals() is recommended when comparing Integer values.

The output of == is not guaranteed to be true for these separately boxed values.

Conclusion: Wrapper classes are useful when object-based values or null values are required. However, care must be taken while comparing wrapper objects and handling null values.

---

5. Alternative Concept and Complexity Analysis

In the original design, fields are declared private and validation is performed through setter methods.

Alternatively, fields can be declared public, and validation can be performed by the calling code.

A. Public Fields Approach

Example:

```
class Vehicle {


   public String regNo;
   public int parkingDuration;
}
```

Object creation:

```
Vehicle v = new Vehicle();


v.regNo = "TN01AB1234";
v.parkingDuration = 120;
```

Here, the variables can be accessed directly without using getters or setters.

However, any class can modify these values without proper validation.

For example:

```
v.parkingDuration = -50;
```

This creates invalid parking information.

B. Problems with Public Fields

1. Any class can directly modify the data.
2. Invalid values can be assigned.
3. Validation must be repeated in different calling classes.
4. Data security and maintainability are reduced.
5. Changes to the internal implementation become difficult.

C. Complexity Analysis

Private Fields with Validating Setters:

- Validation time per update: O(1).
- Validation is implemented in one central location.
- Repeated validation code is avoided.
- Space complexity: O(1) per vehicle for a fixed number of fields.

Public Fields with External Validation:

- Validation time per update: O(1), assuming a fixed number of checks.
- Validation must be repeated wherever the fields are modified.
- Multiple calling classes may require the same validation logic.
- Space complexity: O(1) per vehicle.

If there are m different places modifying the vehicle information, the same validation logic may need to be maintained in up to m places.

Although both approaches can have the same time complexity per update, private fields with setters provide better control and reduce code duplication.

6. Conclusion

Encapsulation protects the data of the Smart-Parking System by using private fields and public getter and setter methods. Packages organize related classes, while access modifiers control visibility. Validation prevents invalid information from entering objects. Wrapper classes and autoboxing help work with primitive values as objects. Therefore, private fields with validating setters are preferred over public fields because they improve security, reliability and maintainability.